



Day 10 Task - Windows Memory Forensics Using Magnet DumpIt & Volatility 3



Trios cyber • 15 Sept (Edited 17 Sept)

90/100 | Due 16 Sept, 18:00

Practical Assignment: Windows Memory Forensics Using Magnet DumpIt & Volatility 3

Dear Students,

For this practical assignment, you are required to acquire volatile memory from your own authorized Windows computer and examine the memory image using Volatility 3 through the Ubuntu WSL terminal.

The objective of this exercise is to understand volatile memory, preserve live system state, acquire a memory image, examine processes, services, loaded modules, user sessions and network connections, and use the findings to support forensic interpretation.

Part 1: Ubuntu WSL Setup

Use Ubuntu WSL for the command-line forensic analysis in this practical.

If Ubuntu WSL is not already installed, follow the official Microsoft documentation:

— Install WSL: <https://learn.microsoft.com/windows/wsl/install>

Open the Ubuntu WSL terminal and confirm that it is working correctly.

Run:

```
— `pwd`  
— `whoami`  
— `uname -a`  
— `lsb_release -a`
```

If `lsb\_release` is not available, install the required package using Ubuntu's package manager.

Record:

```
— Ubuntu distribution/version  
— WSL version, where available
```



- Python version
- Date and time
- Windows operating system

Take screenshots showing the Ubuntu WSL terminal and the results of the commands.

Create a working directory for the practical:

- `mkdir -p ~/memory\_forensics`
- `cd ~/memory\_forensics`

Part 2: Obtain and Set Up Magnet DumpIt

Obtain Magnet DumpIt for Windows from the official Magnet Forensics website:

- Magnet DumpIt: <https://www.magnetforensics.com/resources/magnet-dumpit-for-windows/>

Download the appropriate version for your Windows system and follow the official instructions for running the tool.

Magnet DumpIt is used on the Windows desktop to acquire volatile memory. The analysis will be performed later through Ubuntu WSL.

Record:

- Tool name
- Tool version, where available
- Source from which it was obtained
- Windows operating system
- Date and time of setup

Take screenshots showing:

- Magnet DumpIt obtained and ready for use
- Tool/version information, where available
- Windows desktop before acquisition

Part 3: Create Controlled System Activity

Before acquiring memory, create controlled and authorized activity on the Windows computer.

Perform activities such as:

- Open several applications
- Open a web browser
- Open several browser tabs
- Create a test text file
- Open a command-line application
- Start an application that can be identified in the process list



- Establish a normal network connection
- Log in using your own test account

Record the approximate time at which each activity was performed.

Record:

- Activity performed
- Date/time
- Application used
- User account
- Expected evidence

Take screenshots of selected activities before memory acquisition.

Part 4: Acquire Volatile Memory

Use Magnet DumpIt on the Windows desktop to acquire the physical memory of the authorized Windows computer.

Before starting the acquisition, record:

- Date/time
- Computer name, where available
- Operating system
- Logged-in user
- Acquisition tool
- Acquisition location
- Reason for acquisition

Start the memory acquisition using Magnet DumpIt.

Allow the acquisition to complete.

Do not modify the resulting memory image after acquisition.

After acquisition, record:

- Memory image filename
- File size
- Acquisition date/time
- Storage location
- Acquisition tool/version

Take screenshots showing the completed acquisition and resulting memory image.

Part 5: Access the Memory Image from Ubuntu WSL

Locate the Windows folder containing the memory image.



Access the Windows filesystem from Ubuntu WSL using the `/mnt/` filesystem.

For example:

- `/mnt/c/Users/Username/Downloads/`
- `/mnt/c/Users/Username/Desktop/`

Use commands such as:

- `ls`
- `cd`
- `pwd`
- `file`

Locate the memory image without modifying it.

If the memory image is located on the Windows desktop, navigate to the corresponding WSL path.

Record:

- Original Windows path
- WSL path
- Memory image filename
- File size

Take screenshots showing the memory image being located from Ubuntu WSL.

Part 6: Calculate the Memory Image Hash

Use Ubuntu WSL to calculate a cryptographic hash of the memory image.

Use a command such as:

- `sha256sum <memory\_image>`

Record:

- Hash algorithm
- Memory image filename
- Hash value
- Date/time
- WSL environment used

Take a screenshot of the hash output.

Do not modify the memory image after calculating the hash.

Part 7: Install Python and Volatility 3 in Ubuntu WSL



Check whether Python 3 is installed:

— `python3 --version`

If Python 3 is not installed, install it using the Ubuntu package manager.

Install the required Python tools.

Create a virtual environment for Volatility 3:

— `python3 -m venv ~/volatility3-env`

Activate the environment:

— `source ~/volatility3-env/bin/activate`

Obtain Volatility 3 from the official Volatility Foundation repository:

— Volatility 3: <https://github.com/volatilityfoundation/volatility3>

Install Volatility 3 using the Ubuntu WSL terminal.

Confirm the installation by running:

— `vol -h`

If the `vol` command is not available, use the appropriate Volatility 3 command provided by the installation.

Record:

- Python version
- Volatility 3 version
- Installation method
- Installation directory
- Ubuntu WSL version

Take screenshots showing the successful installation and Volatility 3 help output.

Part 8: Identify the Windows Memory Image

Use Volatility 3 to examine the memory image.

Run the Windows information plugin:

— `vol -f <memory\_image> windows.info`

Record:



- Operating-system information
- Kernel information
- Architecture, where available
- Other relevant information returned by the plugin

Take a screenshot of the relevant terminal output.

If Volatility 3 cannot identify the image, record the error message and investigate the problem instead of claiming that the image is valid.

Part 9: Examine Running Processes and Services

Use Volatility 3 to examine running processes.

Use:

- ``windows.pslist``
- ``windows.pstree``

For example:

- ``vol -f <memory_image> windows.pslist``
- ``vol -f <memory_image> windows.pstree``

Examine:

- Process names
- Process IDs
- Parent/child relationships
- Process information, where available
- User/context information, where available
- Unusual or unexpected processes

Examine available service information using the appropriate Volatility 3 plugin.

Record at least 5 relevant process findings.

For each finding, record:

- Process name
- Process ID
- Parent process, where available
- User/context information, where available
- Observed behavior or relationship
- Why it may be relevant

Take clear screenshots of the terminal output.

Part 10: Examine Loaded Modules and User Sessions



Use appropriate Volatility 3 plugins to examine loaded modules or DLL information associated with processes.

Review:

- Process
- Module/DLL name
- Path, where available
- Associated process
- Other relevant information

Examine available user/session information.

Record:

- User/session information
- Relevant process
- Module/DLL
- Path, where available
- What you observed
- Why it may be relevant

An unexpected module or process should be treated as an investigative lead, not automatically as proof of malware.

Take screenshots of the relevant output.

Part 11: Examine Network Connections

Use an appropriate Volatility 3 Windows network plugin to examine network artifacts available in memory.

Review:

- Local address
- Local port
- Remote address
- Remote port
- Connection state
- Associated process, where available

Record at least 3 relevant network findings, where available.

For each finding, record:

- Process
- Local endpoint
- Remote endpoint
- Connection state
- Other available context



- What you observed
- Why it may be relevant

Take screenshots of the relevant terminal output.

If network information is not available, mention “Not Available.”

Part 12: Investigate a Potentially Suspicious Process

Select ONE process that requires further examination based on the available memory evidence.

Examine:

- Process name
- Process ID
- Parent process
- Process relationship
- Command-line information, where available
- Loaded modules
- Network connections
- User/session context
- Other relevant memory artifacts

Explain whether the available evidence supports further investigation.

Do not label a process as malware solely because its name appears unusual.

Clearly distinguish between:

- Direct evidence/observation
- Inference or interpretation

Take screenshots supporting your conclusion.

Part 13: Timeline & Activity Reconstruction

Use your controlled activity records and memory findings to create a chronological sequence of events.

Your timeline should contain at least 10 events, where sufficient information is available.

For each event, record:

- Date/time
- Artifact/source
- Observed activity
- Interpretation

Compare your known activities with the memory findings.



Identify whether the memory evidence supports the activity you intentionally created.

Clearly distinguish between:

- Direct evidence/observation
- Inference or interpretation

Part 14: Document Your Work

Prepare a PDF or DOCX report containing:

1. Student Information
2. Objective
3. Windows system information
4. Ubuntu WSL information
5. Magnet DumpIt information
6. Memory acquisition details
7. Memory image information
8. Memory image hash
9. Python and Volatility 3 setup
10. Processes and process relationships
11. Services
12. Loaded modules/DLLs
13. User/session information
14. Network connections
15. Suspicious-process examination
16. Screenshots
17. Timeline
18. Direct observations vs inferences
19. Conclusion

Each screenshot should have a short caption explaining what it demonstrates.

Submission Requirements

Submit ONE file only:

- PDF (.pdf)
- Microsoft Word document (.docx)

Important Notes

Use only your own authorized Windows computer for memory acquisition.

Do not acquire memory from another person's computer.

Do not intentionally introduce malware or malicious software into the system.

Do not modify the original memory image after acquisition.



Keep a copy of the original memory image separate from your working files.

Use Ubuntu WSL for the command-line analysis.

Do not use Windows PowerShell for this practical.

If an artifact is unavailable, clearly state "Not Available."

Your conclusion must be based on the available evidence. Do not state that a process is malicious unless the available evidence supports that conclusion.

Expected Learning Outcomes

After completing this practical, you should be able to:

1. Explain why RAM is volatile evidence.
2. Acquire a Windows memory image using an appropriate acquisition tool.
3. Access and analyze the memory image from Ubuntu WSL.
4. Install and use Volatility 3.
5. Identify processes and process relationships.
6. Examine services, loaded modules and user sessions.
7. Examine network connections available in memory.
8. Identify memory artifacts that require further investigation.
9. Correlate memory findings with known system activity.
10. Distinguish direct observations from forensic inferences.
11. Present a traceable memory-forensics examination.

Submission: Upload your completed PDF/DOCX report to this Google Classroom assignment before the specified deadline. Late submissions are allowed only till after 48 hours from the time the task was posted but it may result in deduction of marks.

Please read the instructions carefully before beginning the assignment.

Your work

Marked

Day10.pdf
PDF



[+ Add or create](#)

Resubmit

Your teacher is not accepting work at this time

